Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	TE	Semester:	VI
Course Code:	CSL601	Course Name:	Data Analytics and Visualization Lab

Name of Student:	**Shravani S Raut**
Roll No.:	**40**
Experiment No.:	**05**
Title of the Experiment:	**Implementation of ARIMA model in python**
Date of Performance:	**24/02/2026**
Date of Submission:	**10/03/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Meet Expect Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty: Ms. Odilia Gonsalves

Signature:

Date:

Experiment No. 5

AIM- Implementation of ARIMA model in Python Programming.

Objective- To implement the ARIMA (Auto-Regressive Integrated Moving Average) model in Python for analyzing and forecasting time series data.

Description

The ARIMA model is a widely used statistical technique for analyzing and forecasting time series data. It combines three components:

- **AR (Auto-Regressive)** – Represents the relationship between an observation and its previous values.
- **I (Integrated)** – Refers to differencing the data to make it stationary.
- **MA (Moving Average)** – Represents the relationship between an observation and past forecast errors.

The ARIMA model is generally written as ARIMA(p, d, q), where:

- **p** = order of the AR part
- **d** = degree of differencing
- **q** = order of the MA part

In this practical, the time series data is first visualized to observe trends and seasonality. Stationarity is checked using plots or statistical tests. If the data is non-stationary, differencing is applied.

Next, ACF (Auto-Correlation Function) and PACF (Partial Auto-Correlation Function) plots are used to determine suitable values of p and q. The ARIMA model is then implemented in Python using the statsmodels library. Model parameters are estimated, and residual diagnostics are performed to ensure that the residuals behave like white noise.

Finally, the fitted model is used to forecast future values, and the forecast results are visualized.

PROCEDURE OF ARIMA MODEL

1. Collect Time Series Data

Obtain the time-dependent data required for analysis.

2. Visualize the Data

Plot the time series to observe trend, seasonality, and patterns.

3. Check Stationarity

Determine whether the series is stationary using plots or statistical tests (e.g., ADF test).

4. Apply Differencing (if required)

Make the series stationary by differencing the data dtimes.

5. Identify Model Orders (p, d, q)

- o Use ACF plot to identify the order of MA (q).
- o Use PACF plot to identify the order of AR (p).
- o Determine dfrom the differencing step.

6. Fit the ARIMA Model

Implement the ARIMA(p, d, q) model using statistical software or Python libraries (e.g., statsmodels).

7. Estimate Model Parameters

Calculate coefficients of AR and MA terms.

8. Diagnostic Checking

Analyze residuals to ensure they behave like white noise (no autocorrelation).

9. Forecast Future Values

Use the fitted ARIMA model to predict future observations.

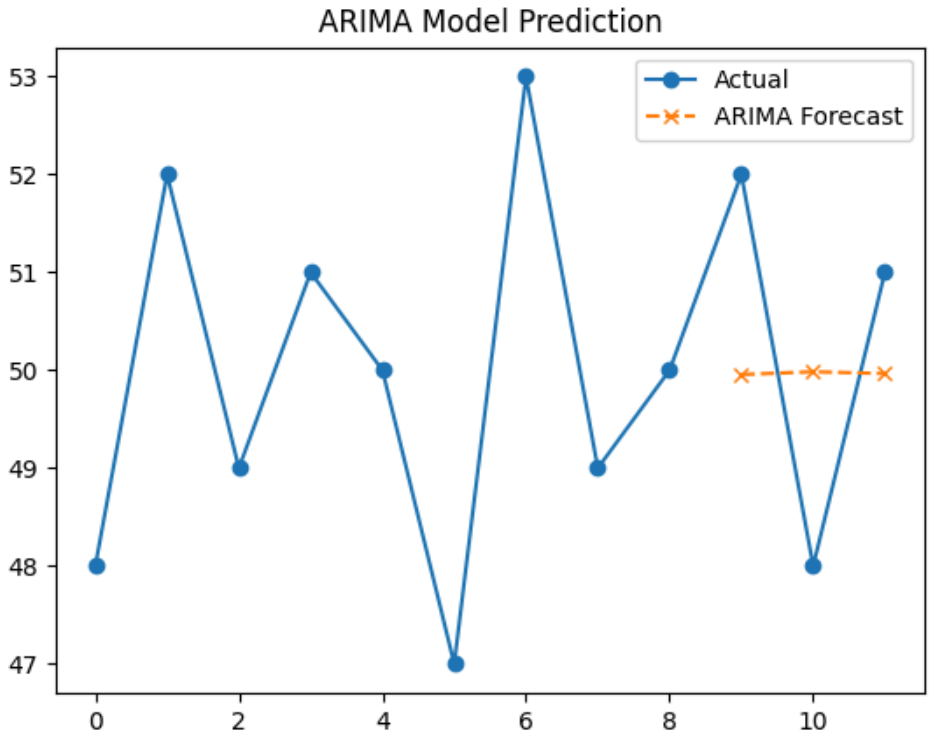
10. Evaluate Model Performance

Assess forecasting accuracy using error measures such as MAE, MSE, or RMSE.

Program: -

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error

# 1. Data
data = [48, 52, 49, 51, 50, 47, 53, 49, 50, 52, 48, 51]
df = pd.Series(data)

# 2. Train-Test Split (last 3 values as test set)
train = df[:-3]
test = df[-3:]

# 3. Fit ARIMA model (p=1, d=1, q=1)
model = ARIMA(train, order=(1, 1, 1))
model_fit = model.fit()

# 4. Forecast
forecast = model_fit.forecast(steps=len(test))

# 5. Accuracy Metrics
mae = mean_absolute_error(test, forecast)
mse = mean_squared_error(test, forecast)
rmse = np.sqrt(mse)

print("ARIMA(1,1,1) Model Results")
print("----------------------------")
print(f"MAE  = {mae:.4f}")
print(f"MSE  = {mse:.4f}")
print(f"RMSE = {rmse:.4f}")

# 6. Plot Results
plt.figure()
plt.plot(df, label='Actual', marker='o')
plt.plot(range(len(train), len(df)), forecast,
        label='ARIMA Forecast', marker='x', linestyle='--')
plt.title("ARIMA Model Prediction")
plt.legend()
plt.show()\

# 7. Accuracy
mape = np.mean(np.abs((test - forecast) / test)) * 100
accuracy = 100 - mape

print(f"MAPE = {mape:.2f}%")
print(f"Accuracy = {accuracy:.2f}%")
```

Output:




```
ARIMA(1,1,1) Model Results
---------------------------
MAE  = 1.6874
MSE  = 3.0613
RMSE = 1.7497
MAPE = 3.36%
Accuracy = 96.64%
```

Conclusion:

The implementation successfully demonstrated how the ARIMA(1, 1, 1) model captures temporal dependencies by combining auto-regression, differencing for stationarity, and moving averages to provide a statistically sound forecast for time-series data.